SHERLOCK

Security Review For Lombard Finance

Collaborative Audit Prepared For: **Lombard Finance**
Lead Security Expert(s): **deadmanwalking**
xiaoming90
Date Audited: **December 15 - December 18, 2025**

Introduction

This update review focused on small updates on the Bascule v1 contract.

Scope

Repository: lombard-finance/smart-contracts

Audited Commit: 19b96a287fc01ca795cabecf5093672ba34d5299

Final Commit: b23211fbfea441d500b57f1e490581a8d2dd4e81

Files:

- contracts/bascule/GMPBasculeV1.sol

Final Commit Hash

b23211fbfea441d500b57f1e490581a8d2dd4e81

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

High Severity Issues: 0 **Medium Severity Issues:** 0 **Low/Info Severity Issues:** 4

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: Incorrect zero-address check [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/9

Vulnerability Detail

It was observed that the zero-address check is not performed against the new `aTrustedSigner`. Instead, the check is done against the previous `trustedSigner`, which is incorrect

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/bascule/GMPBasculeV1.sol#L168

```
File: GMPBasculeV1.sol
165:     function setTrustedSigner(
166:         address aTrustedSigner
167:     ) external onlyRole(DEFAULT_ADMIN_ROLE) {
168:         if (trustedSigner == address(0)) {
169:             revert ZeroTrustedAddress();
170:         }
171:         trustedSigner = aTrustedSigner;
172:         emit TrustedSignerUpdated(aTrustedSigner);
173:     }
```

Impact

If the `trustedSigner` is accidentally set to `address(0)`, the `GMPBasculeV1` will be DOSed as `_checkProof()` function will always revert as it is not possible for `address(0)` to be a signer.

```
File: GMPBasculeV1.sol
283:     function _checkProof(
..SNIP..
295:         // if signer doesn't match consider data invalid
296:         if (signer != trustedSigner) {
297:             return false;
298:         }
299:         return true;
300:     }
```

Code Snippet

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/bascule/GMPBasculeV1.sol#L168

Tool Used

Manual Review

Recommendation

Consider making the following change:

```
- if (trustedSigner == address(0)) {
+ if (aTrustedSigner == address(0)) {
```

Discussion

555-andrew

Acknowledged. Fixed here:
https://github.com/lombard-finance/smart-contracts/pull/361/files

Issue L-2: `mintHistory[mintID].msg` might be empty for minted messages [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/10

Vulnerability Detail

If the mint message is not reported, but it is below the validation threshold, the status of the mint message will be updated to `MintState.MINTED` in Line 261 below, and a mint will be executed.

However, it was observed that only `mintHistory[mintID].status` is updated, while `mintHistory[mintID].msg` remains empty. It is recommended to save the `mintMsg` to storage after setting the status to `MINTED` so that it can be referenced off-chain.

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/bascule/GMPBasculeV1.sol#L261

```
File: GMPBasculeV1.sol
234:     function validateMint(
..SNIP..
249:         // Not reported
250:         if (mintMsg.amount >= validateThreshold) {
251:             // We disallow a withdrawal if it's not in the depositHistory and
252:             // the value is above the threshold.
253:             revert MintFailedValidation(mintID, mintMsg.amount);
254:         }
255:         // We don't have the depositID in the depositHistory, and the value of
 ↪  the
256:         // withdrawal is below the threshold, so we allow the withdrawal
 ↪  without
257:         // additional on-chain validation.
258:         //
259:         // Unlike in original Bascule, this contract records withdrawals
260:         // even when the validation threshold is raised.
261:         mintHistory[mintID].status = MintState.MINTED;
262:         emit MintNotValidated(mintID, mintMsg.amount);
263:     }
```

Impact

If off-chain actors fetch all messages where `status=MINTED`, some `mintHistory[mintID].msg` will be populated, while some will remain empty, which might lead to unexpected results due to the discrepancy.

Code Snippet

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/bascule/GMPBasculeV1.sol#L261

Tool Used

Manual Review

Recommendation

Consider populating the `mintHistory[mintID].msg` whenever the status is updated to `MintState.MINTED` to be consistent

```
- mintHistory[mintID].status = MintState.MINTED;
+ mintHistory[mintID] = Mint(mintMsg, MintState.MINTED);
```

Discussion

555-andrew

I discussed this issue internally and came to conclusion that we should not save mint message to the storage in any cases. This will probably be a new version: `GMPBasculeV2`. Unless you have objections or think we should keep mint message.

555-andrew

Here is the pull request with `GMPBasculeV2`:
https://github.com/lombard-finance/smart-contracts/pull/363

Issue L-3: Mismatch between documented `validateThreshold` modification procedure and implementation [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/12

Summary

The natspec in the constructor of `GMPBasculeV1` states that increasing the `validateThreshold` variable needs to happen in 2 steps where the validation guardian role also needs to be renounced when the threshold is raised. This is not the case however in the actual `updateValidateThreshold` which just updates the state.

Vulnerability Detail

In the constructor of `GMPBasculeV1` the natspec states the following:

```
// By default, the bascule validates all mints and does not grant
// anyone the guardian role. This means that increasing the threshold (or
// turning off validation) requires two steps: (1) grant role and (2) change
// threshold. To preserve this invariant, we renounce the validation
// guardian role when the threshold is raised.
//
// Initialize explicitly for readability/maintainability
validateThreshold = 0; // validate all
```

The implementation however never affects any roles:

```
function updateValidateThreshold(
    uint256 newThreshold
) public onlyRole(VALIDATION_GUARDIAN_ROLE) whenNotPaused {
    // Retains the original reverting behavior of the original
    // for compatibility with off-chain code.
    if (newThreshold == validateThreshold) {
        revert SameValidationThreshold();
    }
    // Actually update the threshold
    _updateValidateThreshold(newThreshold);
}
```

This is likely carryover of previous bascule implementations which needs to be changed.

Impact

Mismatch between documentation and implementation creates ambiguity on corectness.

Code Snippet

-

Tool Used

Manual Review

Recommendation

Remove the outdated natspec in the constructor

Discussion

555-andrew

Acknowledged. Removed outdated natspec here: https://github.com/lombard-finance/smart-contracts/pull/361/files

Issue L-4: Incorrect comments [RESOLVED]

Source: https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/issues/14

Vulnerability Detail

It was observed that the comments in the following are incorrect:

Instance 1 - GMPBasculeV1.sol

```
- // Maximum number of batch deposits it's possible to make at once
+ // Maximum number of batch mints it's possible to make at once
```

```
- // We disallow a withdrawal if it's not in the depositHistory and
+ // We disallow a mint if it's not in the mintHistory and
- // We don't have the depositID in the depositHistory, and the value of the
+ // We don't have the mintID in the mintHistory, and the value of the
```

```
- Set the maximum number of deposits that can be reported at once.
+ Set the maximum number of mints that can be reported at once.
...
- New maximum number of deposits that can be reported at once.
+ New maximum number of mints that can be reported at once.
```

Impact

N/A

Code Snippet

https://github.com/sherlock-audit/2025-12-lombard-wrapper-bascule-dec-15th/blob/04bc3572567ae62455d03c2843c2e991499dda50/smart-contracts/contracts/bascule/GMPBasculeV1.sol#L21

Tool Used

Manual Review

Recommendation

Consider updating the comments to ensure that they are accurate.

Discussion

555-andrew

Acknowledged. Fixed here:
https://github.com/lombard-finance/smart-contracts/pull/361/files

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.